# VISVESVARAYA TECHNOLOGICAL UNIVERSITY

**"JnanaSangama", Belgaum -590014, Karnataka.**

**LAB REPORT**
**on**

# OPERATING SYSTEMS

(23CS4PCOPS)

***Submitted by***

**SHASHANK SHANTHARAM NAYAK (1BM23CS313)**

***in partial fulfillment for the award of the degree of***
**BACHELOR OF ENGINEERING**

***in***
**COMPUTER SCIENCE AND ENGINEERING**

**B.M.S. COLLEGE OF ENGINEERING**
**(Autonomous Institution under VTU)**
**BENGALURU-560019**
**Feb-2025 to June-2025**

**B. M. S. College of Engineering,**

**Bull Temple Road, Bangalore 560019** (Affiliated To Visvesvaraya Technological University, Belgaum) **Department of Computer Science and Engineering**

## CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by SHASHANK SHANTHARAM NAYAK (1BM23CS313)**,** who is bonafide student of **B. M. S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2024. The Lab report has been approved as it satisfies the academic requirements in respect of a **OPERATING SYSTEMS - (23CS4PCOPS)** work prescribed for the said degree.

Amruta B.

Assistant Professor
Department of CSE,
BMSCE, Bengaluru

**Dr. Kavitha Sooda**

Professor and Head Department of CSE

BMSCE, Bengaluru

# Index Sheet

**Course Outcome**

| | |
|---|---|
| CO1 | Apply the different concepts and functionalities of Operating System |
| CO2 | Analyze various Operating system strategies and techniques |
| CO3 | Demonstrate the different functionalities of Operating System |
| CO4 | Conduct practical experiments to implement the functionalities of Operating system |

# Program - 1

## Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)

a) FCFS

b) SJF

## Code:

```
#include <stdio.h>

#include <stdbool.h>


typedef struct process process;

struct process{

        int pid;

        int at;

        int bt;

        int ct;

        int tat;

        int wt;

};


void sort(process* p, int n)

{

        int temp = 0;

        for(int i=0;i<n;i++)
```

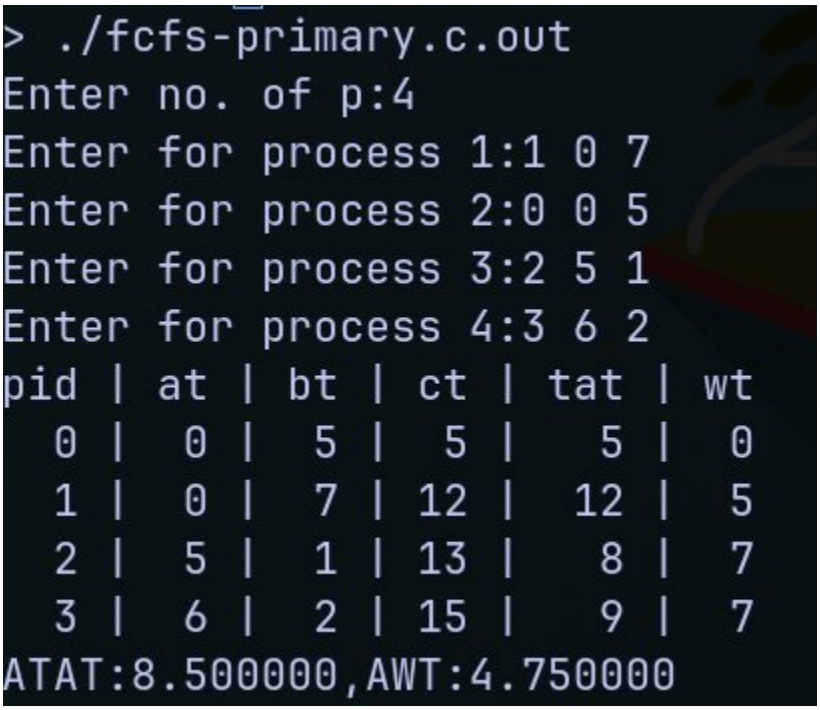
```
{
        for(int j=i;j<n;j++)
        {
                if(p[j].at<p[i].at)
                {
                        temp=p[i].pid;
                        p[i].pid=p[j].pid;
                        p[j].pid=temp;

                        temp=p[i].at;
                        p[i].at=p[j].at;
                        p[j].at=temp;

                        temp=p[i].bt;
                        p[i].bt=p[j].bt;
                        p[j].bt=temp;
                }
                else if(p[j].at==p[i].at)
                {
                        if(p[j].pid<p[i].pid)
                        {
                                temp=p[i].pid;
                                p[i].pid=p[j].pid;
```

```
                                        p[j].pid=temp;

                                        temp=p[i].at;
                                        p[i].at=p[j].at;
                                        p[j].at=temp;

                                        temp=p[i].bt;
                                        p[i].bt=p[j].bt;
                                        p[j].bt=temp;
                                }
                        }
                }
        }
}

void main()
{
        int c=0;
        int n;
        printf("Enter no. of p:");
        scanf("%d",&n);

        process p[n];
```

```
for(int i=0;i<n;i++)
{
        printf("Enter for process %d:",(i+1));
        scanf("%d %d %d", &p[i].pid, &p[i].at, &p[i].bt);
}

sort(p,n);

for(int i=0;i<n;i++)
{
        if(c>=p[i].at)
                c+=p[i].bt;
        else
                c+=p[i].at+p[i].bt-p[i-1].ct;
        p[i].ct = c;
}

for(int i=0;i<n;i++)
{
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
}
```

```
        double avgTAT=0, avgWT=0;

        printf("pid | at | bt | ct | tat | wt\n");

        for (int i = 0; i < n; i++) {

                printf("%3d | %2d | %2d | %2d | %3d | %2d\n", p[i].pid, p[i].at,p[i].bt, p[i].ct,
p[i].tat, p[i].wt);

                avgTAT += p[i].tat;

                avgWT += p[i].wt;

        }

        printf("ATAT:%f,AWT:%f\n",avgTAT/n,avgWT/n);

}
```

## Result:




## Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct process process;
struct process{
        int pid;
        int at;
        int bt;
        int ct;
        int tat;
        int wt;
};

void sjf(process* processes, int n) {
   int currentTime = 0;



        process temp;
        for(int i=0;i<n;i++)
        {
                for(int j=i;j<n;j++)
                {
```

```
                        if((processes[j].at < processes[i].at)||(processes[j].at == processes[i].at
&& processes[j].bt < processes[i].bt))
                        {
                                temp = processes[i];
                                processes[i] = processes[j];
                                processes[j] = temp;
                        }
                }
        }
        for(int i=0;i<n;i++)
        {
                currentTime += processes[i].bt;
                processes[i].ct = currentTime;

        processes[i].tat = processes[i].ct - processes[i].at;  // TAT = CT - AT
              processes[i].wt = processes[i].tat - processes[i].bt;  // WT = Tat - BT
        }

}

void main()
{
        int n;
        printf("Enter n:");
```

```
    scanf("%d",&n);
    process* processes = (process*)malloc(n*sizeof(process));
    for(int i=0;i<n;i++)
    {
        printf("Enter data:");
        scanf("%d%d%d",&processes[i].pid,&processes[i].at,&processes[i].bt);
    }

    sjf(processes, n);

    double avgTAT=0, avgWT=0;
    printf("pid | at | bt | ct | tat | wt\n");
    for (int i = 0; i < n; i++) {
        printf("%3d | %2d | %2d | %2d | %3d | %2d\n", processes[i].pid,
processes[i].at,processes[i].bt, processes[i].ct, processes[i].tat, processes[i].wt);
        avgTAT += processes[i].tat;
        avgWT += processes[i].wt;
    }
    printf("ATAT:%f,AWT:%f\n",avgTAT/n,avgWT/n);
}
```

**Result:**

```
> ./sjf-non.c.out
Enter n:5
Enter data:1 0 8
Enter data:2 0 4
Enter data:3 2 9
Enter data:4 3 5
Enter data:5 4 2
pid | at | bt | ct | tat | wt
  2 |  0 |  4 |  4 |   4 |  0
  1 |  0 |  8 | 12 |  12 |  4
  3 |  2 |  9 | 21 |  19 | 10
  4 |  3 |  5 | 26 |  23 | 18
  5 |  4 |  2 | 28 |  24 | 22
ATAT:16.400000,AWT:10.800000
```

## Program - 2

## Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

## Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <stdbool.h>


typedef struct process process;

struct process{

        int pid;

        int pri;

        int at;

        int bt;

        int ct;

        int tat;

        int wt;

};


void sort(process* p, int n)

{
```

```
        process temp;
        for(int i=0;i<n;i++)
        {
                for(int j=i+1;j<n;j++)
                {
                        if(p[j].at<p[i].at)
                        {
                                temp = p[j];
                                p[j] = p[i];
                                p[i] = temp;
                        }
                }
        }
}

void print(process* p, int n)
{

        double avgTAT=0, avgWT=0;
        printf("pid | at | bt | ct | tat | wt\n");
        for (int i = 0; i < n; i++) {
                p[i].tat = p[i].ct - p[i].at;
                p[i].wt = p[i].tat - p[i].bt;
```

```
            printf("%3d | %2d | %2d | %2d | %3d | %2d\n", p[i].pid, p[i].at,p[i].bt, p[i].ct,
p[i].tat, p[i].wt);

            avgTAT += p[i].tat;

            avgWT += p[i].wt;

        }

        printf("ATAT:%f,AWT:%f\n",avgTAT/n,avgWT/n);

}



void fcfs(process* p, int n, int* ctp)

{

        int c = *ctp;

        for (int i = 0; i < n; i++)

        {

            if (c < p[i].at)

            {

                c = p[i].at;

            }

            c += p[i].bt;

            p[i].ct = c;

        }


        *ctp = c;

}
```

```
void main()
{
	int* ctp =  (int*) malloc(sizeof(int));
	*ctp = 0;
	int n1, n2;
	printf("Enter n1, n2:");
	scanf("%d%d",&n1, &n2);

	process sp[n1];
	process up[n2];
	printf("SYS\n");
	for(int i=0;i<n1;i++)
	{
		printf("Enter the data:");
		scanf("%d%d%d",&sp[i].pid,  &sp[i].at, &sp[i].bt);
		sp[i].pri=0;
	}

	printf("USR\n");
	for(int i=0;i<n2;i++)
	{
		printf("Enter the data:");
		scanf("%d%d%d",&up[i].pid,  &up[i].at, &up[i].bt);
		up[i].pri=1;
```

```
        }
        sort(sp,n1);
        sort(up, n2);

        fcfs(sp, n1, ctp);
        fcfs(up, n2, ctp);

        print(sp,n1);
        print(up,n2);
}
```

## Result:

```
4-04-03-25$ ./multilevel.c.out
Enter n1, n2:3 3
SYS
Enter the data:0 0 2
Enter the data:1 0 6
Enter the data:2 4 7
USR
Enter the data:0 0 8
Enter the data:1 4 9
Enter the data:2 4 9
pid | at | bt | ct | tat | wt
  0 |  0 |  2 |  2 |   2 |  0
  1 |  0 |  6 |  8 |   8 |  2
  2 |  4 |  7 | 15 |  11 |  4
ATAT:7.000000,AWT:2.000000
pid | at | bt | ct | tat | wt
  0 |  0 |  8 | 23 |  23 | 15
  1 |  4 |  9 | 32 |  28 | 19
  2 |  4 |  9 | 41 |  37 | 28
ATAT:29.333333,AWT:20.666667
```

## Program - 3

## Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms

a) Rate- Monotonic

## Code:

```
#include <stdio.h>


#define MAX_PROCESSES 10

#define SIMULATION_TIME 30  // Total simulation time in time units


typedef struct {

   int id;

   int period;

   int exec_time;

   int next_release;

   int remaining_time;

   int active;  // Indicates if the process is ready to run

} process;


int main() {

   process processes[MAX_PROCESSES];

   int n;
```

```
printf("Number of processes: ");
scanf("%d", &n);

for (int i = 0; i < n; i++) {
    processes[i].id = i;
    printf("Process %d period: ", i);
    scanf("%d", &processes[i].period);
    printf("Process %d execution time: ", i);
    scanf("%d", &processes[i].exec_time);
    processes[i].next_release = 0;
    processes[i].remaining_time = 0;
    processes[i].active = 0;
}

// Sort by period (Rate Monotonic Priority)
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (processes[i].period > processes[j].period) {
            process temp = processes[i];
            processes[i] = processes[j];
            processes[j] = temp;
        }
    }
```

```
}

printf("\n--- Rate Monotonic Scheduling Simulation ---\n\n");

// Simulation loop
for (int time = 0; time < SIMULATION_TIME; time++) {

   // Check for new releases
   for (int i = 0; i < n; i++) {
      if (time == processes[i].next_release) {
         if (processes[i].remaining_time > 0) {
            printf("Time %d: Process %d MISSED DEADLINE\n", time, processes[i].id);
         }

         processes[i].next_release += processes[i].period;
         processes[i].remaining_time = processes[i].exec_time;
         processes[i].active = 1;
      }
   }

   // Run highest-priority active process
   int ran = 0;
   for (int i = 0; i < n; i++) {
      if (processes[i].active && processes[i].remaining_time > 0) {
```

```
            if (processes[i].remaining_time == processes[i].exec_time)
                printf("Time %d: Process %d START\n", time, processes[i].id);

            processes[i].remaining_time--;

            if (processes[i].remaining_time == 0) {
                processes[i].active = 0;
                printf("Time %d: Process %d END\n", time, processes[i].id);
            }
            ran = 1;
            break;  // Preempt lower-priority processes
        }
    }

    if (!ran) {
        printf("Time %d: IDLE\n", time);
    }
  }

  return 0;
}
```

**Result:**

```
> ./rms
Number of processes: 2
Process 0 period: 4
Process 0 execution time: 1
Process 1 period: 6
Process 1 execution time: 2

--- Rate Monotonic Scheduling Simulation ---

Time 0: Process 0 START
Time 0: Process 0 END
Time 1: Process 1 START
Time 2: Process 1 END
Time 3: IDLE
Time 4: Process 0 START
Time 4: Process 0 END
Time 5: IDLE
Time 6: Process 1 START
Time 7: Process 1 END
Time 8: Process 0 START
Time 8: Process 0 END
Time 9: IDLE
Time 10: IDLE
Time 11: IDLE
```

## Program - 4

## Question:

Write a C program to simulate:

a) Producer-Consumer problem using semaphores.

b) Dining-Philosopher's problem

## Code:

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>


#define N 5

int full = 0;

int empty = N;

int mutex = 1;

int BUFFER[N];

int in = 0; // Index for producing

int out = 0; // Index for consuming


void wait(int* s) {

   while (*s <= 0);

   (*s)--;

}
```

```
void signal(int* s) {
   (*s)++;
}

void produce(int item) {
   wait(&empty);
   wait(&mutex);
   BUFFER[in] = item;
   in = (in + 1) % N;
   printf("Produced: %d\n", item);
   signal(&mutex);
   signal(&full);
}

void consume() {
   wait(&full);
   wait(&mutex);
   int item = BUFFER[out];
   out = (out + 1) % N;
   printf("Consumed: %d\n", item);
   signal(&mutex);
   signal(&empty);
}
```

```
void* produce_thread(void* arg) {
    int item = 1;
    while (1) {
        produce(item);
        item++;
        sleep(1);
    }
    return NULL;
}

void* consume_thread(void* arg) {
    while (1) {
        consume();
        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t producer, consumer;

    pthread_create(&producer, NULL, produce_thread, NULL);
    pthread_create(&consumer, NULL, consume_thread, NULL);
```

```
    pthread_join(producer, NULL);
    pthread_join(consumer, NULL);

    return 0;
}
```

## Result:

```
> ./prod-cons
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 3
Produced: 6
Consumed: 4
Produced: 7
Produced: 8
Consumed: 5
Produced: 9
Produced: 10
Consumed: 6
Produced: 11
Consumed: 7
Produced: 12
^C
```

## Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5  // Number of philosophers

// Define a semaphore structure to maintain each fork's state
struct semaphore {
   int mutex_value;       // 1 if available, 0 if not
   pthread_t* waiting_list; // List of threads waiting for the fork
};

// Declare semaphores for each fork
struct semaphore forks[N];

// Declare the philosophers' threads
pthread_t philosophers[N];

// Function declarations
void* philosopher(void* arg);
void think(int id);
void eat(int id);
void take_forks(int id);
```

```
void put_forks(int id);
void wait(struct semaphore* s);
void signal(struct semaphore* s);

int main() {
   // Initialize semaphores for each fork
   for (int i = 0; i < N; i++) {
      forks[i].mutex_value = 1; // All forks are initially available
      forks[i].waiting_list = NULL; // No one is waiting initially
   }

   // Create philosopher threads
   for (int i = 0; i < N; i++) {
      pthread_create(&philosophers[i], NULL, philosopher, (void*)(long)i);
   }

   // Join philosopher threads (they run forever in this case)
   for (int i = 0; i < N; i++) {
      pthread_join(philosophers[i], NULL);
   }

   return 0;
}
```

```
void* philosopher(void* arg) {
    int id = (int)(long)arg;
    while (1) {
        think(id);
        take_forks(id);
        eat(id);
        put_forks(id);
    }
}

void think(int id) {
    printf("Philosopher %d is thinking.\n", id);
    sleep(1);  // Thinking for some time
}

void eat(int id) {
    printf("Philosopher %d is eating.\n", id);
    sleep(2);  // Eating for some time
}

void take_forks(int id) {
    // Wait for the left fork
    wait(&forks[id]);
```

```
    // Wait for the right fork (next fork in circular manner)
    wait(&forks[(id + 1) % N]);

    printf("Philosopher %d took both forks.\n", id);
}

void put_forks(int id) {
    // Release the left fork
    signal(&forks[id]);

    // Release the right fork (next fork in circular manner)
    signal(&forks[(id + 1) % N]);

    printf("Philosopher %d put down both forks.\n", id);
}

void wait(struct semaphore* s) {
    // If the semaphore is not available, block the philosopher and wait
    while (s->mutex_value <= 0) ;
    s->mutex_value = 0;  // Fork is now taken by the philosopher
}

void signal(struct semaphore* s) {
    s->mutex_value = 1;  // Fork is now available
```

}

**Result:**

```
> ./dine
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 took both forks.
Philosopher 0 is eating.
Philosopher 4 took both forks.
Philosopher 4 is eating.
Philosopher 0 put down both forks.
Philosopher 0 is thinking.
Philosopher 3 took both forks.
Philosopher 3 is eating.
Philosopher 4 put down both forks.
Philosopher 4 is thinking.
Philosopher 2 took both forks.
Philosopher 2 is eating.
Philosopher 3 put down both forks.
Philosopher 3 is thinking.
Philosopher 1 took both forks.
Philosopher 1 is eating.
Philosopher 2 put down both forks.
Philosopher 2 is thinking.
^C
```

## Program - 5

## Question:

Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance.

## Code:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
   int P, R;

   printf("Enter number of processes: ");
   scanf("%d", &P);
   printf("Enter number of resource types: ");
   scanf("%d", &R);

   int Allocation[P][R], Max[P][R], Need[P][R], Available[R];
   bool Finish[P];
   int Work[R], SafeSequence[P];

   // Input Allocation matrix
   printf("Enter Allocation matrix (%d x %d):\n", P, R);
   for (int i = 0; i < P; i++)
```

```
    for (int j = 0; j < R; j++)
        scanf("%d", &Allocation[i][j]);

// Input Max matrix
printf("Enter Max matrix (%d x %d):\n", P, R);
for (int i = 0; i < P; i++)
    for (int j = 0; j < R; j++)
        scanf("%d", &Max[i][j]);

// Input Available vector
printf("Enter Available resources (%d):\n", R);
for (int i = 0; i < R; i++)
    scanf("%d", &Available[i]);

// Calculate Need matrix = Max - Allocation
for (int i = 0; i < P; i++)
    for (int j = 0; j < R; j++)
        Need[i][j] = Max[i][j] - Allocation[i][j];

// Initialize
for (int i = 0; i < R; i++)
    Work[i] = Available[i];
for (int i = 0; i < P; i++)
    Finish[i] = false;
```

```
int count = 0;

while (count < P) {
   bool found = false;
   for (int i = 0; i < P; i++) {
      if (!Finish[i]) {
         bool canRun = true;
         for (int j = 0; j < R; j++) {
            if (Need[i][j] > Work[j]) {
               canRun = false;
               break;
            }
         }

         if (canRun) {
            for (int j = 0; j < R; j++)
               Work[j] += Allocation[i][j];
            SafeSequence[count++] = i;
            Finish[i] = true;
            found = true;
         }
      }
   }
```

```
        if (!found) {
            printf("System is NOT in a safe state.\n");
            return 1;
        }
    }

    printf("System is in a safe state.\nSafe sequence: ");
    for (int i = 0; i < P; i++)
        printf("P%d ", SafeSequence[i]);
    printf("\n");

    return 0;
}
```

## Result:

```
> ./banker
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation matrix (5 x 3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max matrix (5 x 3):
7 5 3
3 2 2
9 0 2
4 2 2
5 3 3
Enter Available resources (3):
3 3 2
System is in a safe state.
Safe sequence: P1 P3 P4 P0 P2
```

## Program - 6

## Question:

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit

b) Best-fit

c) First-fit

## Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <stdbool.h>

#include <time.h>


typedef struct part part;

struct part {

   int size;

   int pro;

   part* next;

};


part* create() {

   part* node = (part*)malloc(sizeof(part));

   node->size = 0;

   node->next = NULL;
```

```
    node->pro = -1;  // -1 means the partition is free
    return node;
}

void mempart(part* ptr, int pro, int size) {
    part* npart = create();
    npart->size = ptr->size - size;
    npart->next = ptr->next;
    ptr->pro = pro;
    ptr->size = size;
    ptr->next = npart;
}

void worst_fit(part* head, int pro, int size) {
    part* temp = head;
    part* largest = NULL;
    while (temp != NULL) {
        if (temp->pro == -1 && temp->size >= size) {
            if (largest == NULL || temp->size > largest->size)
                largest = temp;
        }
        temp = temp->next;
    }
    if (largest != NULL) {
```

```
        mempart(largest, pro, size);
    }
}

void first_fit(part* head, int pro, int size) {
    part* temp = head;
    while (temp != NULL) {
        if (temp->pro == -1 && temp->size >= size) {
            mempart(temp, pro, size);
            break;
        }
        temp = temp->next;
    }
}

void best_fit(part* head, int pro, int size) {
    part* temp = head;
    part* best = NULL;
    while (temp != NULL) {
        if (temp->pro == -1 && temp->size >= size) {
            if (best == NULL || temp->size < best->size)
                best = temp;
        }
        temp = temp->next;
```

```
    }
    if (best != NULL) {
        mempart(best, pro, size);
    }
}

void printPart(part* head) {
    part* temp = head;
    while (temp != NULL) {
        if (temp->pro == -1) {
            printf("Empty:%d ", temp->size);
        } else {
            printf("P%d:%d ", temp->pro, temp->size);
        }
        temp = temp->next;
    }
    printf("\n");
}

part* clone(part* head) {
    if (head == NULL) {
        return NULL;
    }
```

```
    part* newHead = create();
    newHead->size = head->size;
    newHead->pro = head->pro;
    part* temp = head->next;
    part* newTemp = newHead;

    while (temp != NULL) {
        part* newNode = create();
        newNode->size = temp->size;
        newNode->pro = temp->pro;
        newTemp->next = newNode;
        newTemp = newNode;
        temp = temp->next;
    }

    return newHead;
}

// Random initial memory fragmentation and allocation
void random_allocate(part* head, int n, int p[]) {
    int total_size;
    printf("Enter total memory size: ");
    scanf("%d", &total_size);
```

```
int remaining = total_size;
head->size = 0; // Dummy head; will replace later
part* current = head;

printf("Randomly allocating initial memory...\n");

while (remaining > 0) {
   int split_size = (rand() % (remaining / 2 + 1)) + 1;
   if (split_size > remaining) split_size = remaining;

   bool assign_process = rand() % 2 == 0;

   part* node = create();
   node->size = split_size;

   if (assign_process && n > 0) {
      int idx = rand() % n;
      if (p[idx] <= split_size) {
         node->pro = idx;
         printf("Allocated P%d of size %d\n", idx, split_size);
      } else {
         node->pro = -1;
         printf("Empty:%d\n", split_size);
      }
```

```
        } else {
            node->pro = -1;
            printf("Empty:%d\n", split_size);
        }

        current->next = node;
        current = node;
        remaining -= split_size;
    }

    // Remove dummy head and promote first real partition
    part* realHead = head->next;
    free(head);
    *head = *realHead;
    free(realHead);
}

int main() {
    srand(time(0));  // Seed for random number generation

    part* head = create();

    int n;
    printf("Enter number of processes: ");
```

```
scanf("%d", &n);
int p[n];
printf("Enter process sizes:\n");
for (int i = 0; i < n; i++) {
   scanf("%d", &p[i]);
}

// Generate random memory with some pre-allocated processes
random_allocate(head, n, p);

while (true) {
   part* clonedHead = clone(head);

   int choice;
   printf("\n1.First Fit 2.Best Fit 3.Worst Fit 4.Exit: ");
   scanf("%d", &choice);

   if (choice == 4) {
      break;
   }

   switch (choice) {
      case 1:
         for (int i = 0; i < n; i++) {
```

```
                first_fit(clonedHead, i, p[i]);
            }
            break;
        case 2:
            for (int i = 0; i < n; i++) {
                best_fit(clonedHead, i, p[i]);
            }
            break;
        case 3:
            for (int i = 0; i < n; i++) {
                worst_fit(clonedHead, i, p[i]);
            }
            break;
        default:
            printf("Invalid choice!\n");
            continue;
    }

    printPart(clonedHead);
  }

  return 0;
}
```

**Result:**

```
> ./fit
Enter number of processes: 4
Enter process sizes:
100
200
300
400
Enter total memory size: 1000
Randomly allocating initial memory...
Empty:93
Allocated P2 of size 345
Empty:159
Allocated P1 of size 102
Allocated P3 of size 225
Empty:76

1.First Fit 2.Best Fit 3.Worst Fit 4.Exit: 1
P2:345 P1:102 P3:225 P0:100 P1:200 P2:300 P3:400 Empty:76

1.First Fit 2.Best Fit 3.Worst Fit 4.Exit: 2
P2:345 P1:102 P3:225 P0:100 P1:200 P2:300 P3:400 Empty:76

1.First Fit 2.Best Fit 3.Worst Fit 4.Exit: 3
P2:345 P1:102 P3:225 P0:100 P1:200 P2:300 P3:400 Empty:76

1.First Fit 2.Best Fit 3.Worst Fit 4.Exit: 4
Exiting...
```

## Program - 7

## Question:

Write a C program to simulate page replacement algorithms.

a) FIFO

b) LRU

c) Optimal

## Code:

```
#ifndef PAGE_REPLACEMENT_H

#define PAGE_REPLACEMENT_H


#include <stdio.h>


// Macro for taking user input

#define GET_INPUT(frames, pages, refString) \

   printf("Enter number of frames: "); \

   scanf("%d", &frames); \

   printf("Enter number of pages: "); \

   scanf("%d", &pages); \

   printf("Enter reference string: "); \

   for (int i = 0; i < pages; i++) { \

      scanf("%d", &refString[i]); \

   }
```

```
// Macro for initializing frames
#define INIT_FRAMES(frames, frame, counter) \
   for (int i = 0; i < frames; i++) { \
      frame[i] = -1; \
      if (counter != NULL) counter[i] = 0; \
   }

// Macro for displaying frame contents
#define DISPLAY_FRAMES(page, frames, frame) \
   printf("%d: ", page); \
   for (int i = 0; i < frames; i++) { \
      printf("%d ", frame[i]); \
   } \
   printf("\n");

// Macro for printing total page faults
#define PRINT_PAGE_FAULTS(pageFaults) \
   printf("Total Page Faults: %d\n", pageFaults);

#endif
```

## Code:

```
#include "page_replacement.h"

int main() {
```

```
int frames, pages, pageFaults = 0, current = 0;
int referenceString[100], frame[100]; // Max size to prevent runtime issues

GET_INPUT(frames, pages, referenceString);

// Initialize frames
for (int i = 0; i < frames; i++) {
        frame[i] = -1;
}

for (int i = 0; i < pages; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
                if (frame[j] == referenceString[i]) {
                        found = 1;
                        break;
                }
        }
        if (!found) {
                frame[current] = referenceString[i];
                current = (current + 1) % frames;
                pageFaults++;
        }
        DISPLAY_FRAMES(referenceString[i], frames, frame);
```

```
        }

        PRINT_PAGE_FAULTS(pageFaults);
        return 0;
}
```

## Code:

```
#include "page_replacement.h"

int main() {
   int frames, pages, pageFaults = 0;
   int referenceString[100], frame[100]; // Frame contents
   int lastUsed[100];                    // Tracks last used time of each frame slot

   GET_INPUT(frames, pages, referenceString);

   // Initialize frames and usage history
   for (int i = 0; i < frames; i++) {
      frame[i] = -1;
      lastUsed[i] = -1;
   }

   for (int time = 0; time < pages; time++) {
      int currentPage = referenceString[time];
      int found = 0;
```

```
// Check if current page is already in frame
for (int j = 0; j < frames; j++) {
   if (frame[j] == currentPage) {
      found = 1;
      lastUsed[j] = time;  // Update usage time
      break;
   }
}

// Page fault
if (!found) {
   int index = -1;

   // Find an empty frame
   for (int j = 0; j < frames; j++) {
      if (frame[j] == -1) {
         index = j;
         break;
      }
   }

   // No empty frame; replace LRU
   if (index == -1) {
```

```
            int minTime = time;
            for (int j = 0; j < frames; j++) {
                if (lastUsed[j] < minTime) {
                    minTime = lastUsed[j];
                    index = j;
                }
            }
        }

        frame[index] = currentPage;
        lastUsed[index] = time;
        pageFaults++;
    }

    DISPLAY_FRAMES(currentPage, frames, frame);
  }

  PRINT_PAGE_FAULTS(pageFaults);
  return 0;
}
```

## Code:

```
#include "page_replacement.h"  // Ensure this header provides: GET_INPUT, DISPLAY_FRAMES, PRINT_PAGE_FAULTS
```

```
int main() {
    int frames, pages, pageFaults = 0;
    int referenceString[100], frame[100];

    GET_INPUT(frames, pages, referenceString);

    // Initialize all frames to -1 (indicating empty)
    for (int i = 0; i < frames; i++) {
        frame[i] = -1;
    }

    // Traverse through each page in the reference string
    for (int i = 0; i < pages; i++) {
        int currentPage = referenceString[i];
        int found = 0;

        // Check if the page is already in one of the frames (page hit)
        for (int j = 0; j < frames; j++) {
            if (frame[j] == currentPage) {
                found = 1;
                break;
            }
        }
```

```
// If not found (page fault)
if (!found) {
    int replaceIndex = -1;

    // First, check for any empty frame
    for (int j = 0; j < frames; j++) {
        if (frame[j] == -1) {
            replaceIndex = j;
            break;
        }
    }

    // If no empty frame, find the page used farthest in future
    if (replaceIndex == -1) {
        int farthest = -1;

        for (int j = 0; j < frames; j++) {
            int nextUse = pages;  // Default: not used again

            // Look ahead in the reference string
            for (int k = i + 1; k < pages; k++) {
                if (frame[j] == referenceString[k]) {
                    nextUse = k;
                    break;
```

```
                }
            }

            // Select the frame with the farthest use
            if (nextUse > farthest) {
                farthest = nextUse;
                replaceIndex = j;
            }
        }
    }

    // Replace the selected frame with the current page
    frame[replaceIndex] = currentPage;
    pageFaults++;
  }

  // Display current frame contents after this page reference
  DISPLAY_FRAMES(currentPage, frames, frame);
 }

 // Output total page faults
 PRINT_PAGE_FAULTS(pageFaults);
 return 0;
}
```

**Result:**

```
> ./main_fifo
Enter number of frames: 3
Enter number of pages: 8
Enter reference string: 7 0 1 2 0 3 0 4
7: 7 -1 -1
0: 7 0 -1
1: 7 0 1
2: 2 0 1
0: 2 0 1
3: 2 3 1
0: 2 3 0
4: 4 3 0
Total Page Faults: 7
```

```
> ./main_lru
Enter number of frames: 3
Enter number of pages: 8
Enter reference string: 7 0 1 2 0 3 0 4
7: 7 -1 -1
0: 7 0 -1
1: 7 0 1
2: 2 0 1
0: 2 0 1
3: 2 0 3
0: 2 0 3
4: 4 0 3
Total Page Faults: 6
```

```
> ./main_optimal
Enter number of frames: 3
Enter number of pages: 8
Enter reference string: 7 0 1 2 0 3 0 4
7: 7 -1 -1
0: 7 0 -1
1: 7 0 1
2: 2 0 1
0: 2 0 1
3: 3 0 1
0: 3 0 1
4: 4 0 1
Total Page Faults: 6
```

## Program - 8

## Question:

Write a C program to simulate the following file allocation strategies.

a) Sequential

b) Indexed

c) Linked

## Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>


#define DISK_SIZE 100


int disk[DISK_SIZE];  // 0 = free, 1 = allocated


// For indexed allocation

typedef struct {

   char name[20];

   int indexBlock;

   int blocks[10];

   int size;

} IndexedFile;
```

```
// For linked allocation
typedef struct Node {
    int block;
    struct Node* next;
} Node;

typedef struct {
    char name[20];
    Node* start;
    int size;
} LinkedFile;

void resetDisk() {
    for (int i = 0; i < DISK_SIZE; i++)
        disk[i] = 0;
}

void sequentialAllocation() {
    char name[20];
    int start, length;

    printf("\n--- Sequential Allocation ---\n");
    printf("Enter file name: ");
    scanf("%s", name);
```

```
    printf("Enter starting block: ");
    scanf("%d", &start);
    printf("Enter length (number of blocks): ");
    scanf("%d", &length);

    if (start + length > DISK_SIZE) {
        printf("Error: File exceeds disk size.\n");
        return;
    }

    for (int i = start; i < start + length; i++) {
        if (disk[i] == 1) {
            printf("Error: Block %d already allocated.\n", i);
            return;
        }
    }

    for (int i = start; i < start + length; i++) {
        disk[i] = 1;
    }

    printf("File '%s' allocated from block %d to %d.\n", name, start, start + length - 1);
}
```

```
void indexedAllocation() {
    IndexedFile file;
    printf("\n--- Indexed Allocation ---\n");
    printf("Enter file name: ");
    scanf("%s", file.name);
    printf("Enter number of blocks needed: ");
    scanf("%d", &file.size);

    printf("Enter index block: ");
    scanf("%d", &file.indexBlock);

    if (disk[file.indexBlock] == 1) {
        printf("Error: Index block already in use.\n");
        return;
    }

    int count = 0;
    for (int i = 0; i < DISK_SIZE && count < file.size; i++) {
        if (disk[i] == 0 && i != file.indexBlock) {
            file.blocks[count++] = i;
        }
    }

    if (count < file.size) {
```

```
        printf("Error: Not enough free blocks.\n");
        return;
    }

    disk[file.indexBlock] = 1;
    for (int i = 0; i < file.size; i++) {
        disk[file.blocks[i]] = 1;
    }

    printf("File '%s' allocated with index block %d and data blocks: ", file.name, file.indexBlock);
    for (int i = 0; i < file.size; i++) {
        printf("%d ", file.blocks[i]);
    }
    printf("\n");
}

void linkedAllocation() {
    LinkedFile file;
    printf("\n--- Linked Allocation ---\n");
    printf("Enter file name: ");
    scanf("%s", file.name);
    printf("Enter number of blocks needed: ");
    scanf("%d", &file.size);
```

```
Node* head = NULL;
Node* current = NULL;
int count = 0;

for (int i = 0; i < DISK_SIZE && count < file.size; i++) {
   if (disk[i] == 0) {
      disk[i] = 1;
      Node* newNode = (Node*)malloc(sizeof(Node));
      newNode->block = i;
      newNode->next = NULL;

      if (head == NULL) {
         head = newNode;
         current = newNode;
      } else {
         current->next = newNode;
         current = newNode;
      }
      count++;
   }
}

if (count < file.size) {
   printf("Error: Not enough free blocks.\n");
```

```
            return;
        }

        file.start = head;

        printf("File '%s' allocated at blocks: ", file.name);
        Node* temp = head;
        while (temp) {
            printf("%d -> ", temp->block);
            temp = temp->next;
        }
        printf("NULL\n");
    }

    int main() {
        int choice;
        while (1) {
            printf("\nFile Allocation Strategies:\n");
            printf("1. Sequential Allocation\n");
            printf("2. Indexed Allocation\n");
            printf("3. Linked Allocation\n");
            printf("4. Reset Disk\n");
            printf("5. Exit\n");
            printf("Enter your choice: ");
```

```
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            sequentialAllocation();
            break;
        case 2:
            indexedAllocation();
            break;
        case 3:
            linkedAllocation();
            break;
        case 4:
            resetDisk();
            printf("Disk reset.\n");
            break;
        case 5:
            printf("Exiting.\n");
            return 0;
        default:
            printf("Invalid choice.\n");
    }
}
```

```
    return 0;
}
```

## Result:

```
> ./file_alloc
File Allocation Strategies:
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Reset Disk
5. Exit
Enter your choice: 1
--- Sequential Allocation ---
Enter file name: FileA
Enter starting block: 5
Enter length (number of blocks): 4
File 'FileA' allocated from block 5 to 8.

File Allocation Strategies:
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Reset Disk
5. Exit
Enter your choice: 2
--- Indexed Allocation ---
Enter file name: FileB
Enter number of blocks needed: 3
Enter index block: 12
File 'FileB' allocated with index block 12 and data block
s: 14 16 18

File Allocation Strategies:
1. Sequential Allocation
```

```
File Allocation Strategies:
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Reset Disk
5. Exit
Enter your choice: 3
--- Linked Allocation ---
Enter file name: FileC
Enter number of blocks needed: 4
File 'FileC' allocated at blocks: 21 -> 23 -> 27 -> 30 ->
 NULL

File Allocation Strategies:
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Reset Disk
5. Exit
Enter your choice: 5
Exiting.
```